

SEVİYE SEVİYE USTALAŞMA

Claude Code Ustalık Yolu

Acemi'den Plus'a: her seviyede ne yaparsın, hangi araçları kullanırsın, skill / agent / hook'u nasıl oluşturursun.

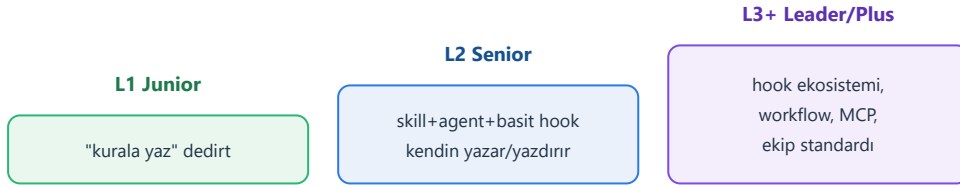


Yetkinlik Haritası — bir bakışta

Her seviye bir öncekini kapsar. "Geçiş kapısı"nı geçmeden üst seviyeye atlama.

Yetkinlik	L0 Acemi	L1 Junior	L2 Senior	L3 Leader	L4 Plus
Oturum aç/kapat, onayla	✓	✓	✓	✓	✓
Komut + skill kullan (/code-check...)	—	✓	✓	✓	✓
Kural/journal/TODO yönet	—	✓	✓	✓	✓
Skill / agent / hook oluştur	—	kısmi	✓	✓	✓
Plan-first (Tier 3), agent çağır	—	—	✓	✓	✓
Template kur/güncelle, ekip standardı	—	—	—	✓	✓
Workflow orkestrasyon, MCP, meta	—	—	—	kısmi	✓

Skill / Agent / Hook oluşturma karmaşıklığı seviyeye göre



Felsefe: Aşağı seviye = aracı *kullanır*. Yukarı seviye = aracı *üretir* ve ekip için *standartlaştırır*. Ustalık, tekrar eden acıyı kalıcı yapıya çevirmektir.

Baştan netleşsin: Bu sistemdeki otomatik kolaylıkların (oturum özeti, plan-onay, otomatik günlük, güvenlik bloğu, commit disiplini) hiçbiri Claude'un hazır sihri değil. Her biri **birinin yazdığı/yazdırıldığı skill, agent ve hook sayesinde** çalışıyor — L0-L1'de hazır bulursun, L2'den sonra **kendin üretirsin**. Yukarı tırmanmak = bu yapının daha çoğunu sen yazmak.

Hangi söz → hangi yapı → ne yapar

Sen dersin	Hangi yapı sayesinde	Ne yapar
günaydın	session-start hook + session-handoff	git/TODO/journal okur, "nerede kaldık" özetler
iyi geceler / /handoff	session-handoff skill	bugünü journale yazar, TODO işaretler, journal+TODO commit'ler
"bunu commit'le"	commit-discipline kuralı	tek-konu, doğru formatlı commit
"dosyaları böl"	commit-splitter agent	15+ dosyayı anlamlı commit bucket'larına ayırır
"hangi seçenek?"	Ilm-council skill	5 danışman + sentez ile karar analizi

/verify	verify skill + test-discipline	uygulamayı çalıştırır, davranışı doğrular
(commit'te şifre)	pre-commit-antipattern hook	sızıntıyı yakalar, commit'i bloklar
(plan gerektiren iş)	plan-first kuralı	kod yazmadan önce plan + onay ister

Kalıp: her kolaylık = **tetik söz** + **senin kurduğun yapı**. Yapıyı sil → kolaylık kaybolur. Yeni yapı yaz → yeni kolaylık.

Hedef: Korkmadan oturum açıp iş yaptırabilmek. Kod bilmek şart değil.

Kazanımlar

- ✓ `cd proje` → `claude` ile aç
- ✓ `günaydın` / `iyi geceler` ritüeli
- ✓ Hedefi net Türkçe söyle, planı onayla
- ✓ "bunu commit'le" ile kayıt noktası
- ✓ Gösterileni oku, körü körüne onaylama

Kullandığın araçlar

`günaydın` `iyi geceler` `/handoff` `"commit'le"`

Skill / agent / hook oluşturma — bu seviyede

Henüz oluşturmuyorsun. Hook/skill/kural zaten kurulu (bootstrap ile geldi) — sen sadece tetikliyorsun. Yaratma L1'de "kurala yaz" ile başlar.

Sorumluluk

Onayladığın şeyden sorumlusun. Anlamadıysan sor: "bu ne yapıyor?"

Geçiş kapısı → L1: Bir haftalık iş akışını yardımsız tamamladıysan, cheat-sheet'teki komutları merak etmeye başladıysan Junior'sın.

Hedef: Hazır komut/skill/kuralları bilinçli kullanmak; hafıza katmanını yönetmek.

Kazanımlar

- ✓ Komutlar: `/code-check`, `/verify`, `/security-check`, `/feature-dev`
- ✓ `/compact` (devam) vs `/clear` (yeni iş) farkı
- ✓ Kuralları okur, neden var anlar; TODO/journal takip eder
- ✓ Commit disiplini: 1 commit = 1 konu, 15 dosya eşiği
- ✓ Tier farkı: trivial mi, plan gerektirir mi (sezgi)

3 katman hafıza — nereye ne yazılır

Bilgi	Yer
Proje kimliği (değişmez)	<code>CLAUDE.md</code>
Kalıcı davranış kuralı	<code>.claude/rules/<konu>.md</code>
Plan / öncelik	<code>TODO.md</code>
Günlük notu	<code>docs/journal/YYYY-MM-DD.md</code>

Skill / agent / hook oluşturma — bu seviyede

Sıfırdan yazmıyorsun ama **kural ürettiriyorsun**. Tekrarlanan bir talimatı kalıcı yap:

```
# Sen:  
bunu kalıcı kurala yaz: e-posta metinleri her zaman Türkçe olacak  
  
# Claude → .claude/rules/<konu>.md dosyasına işler, onayını alır
```

Junior refleksi: Aynı şeyi 2. kez söylüyorsan → "kurala yaz" de. Hafıza dosyada yaşar, konuşmada değil.

Geçiş kapısı → L2: Kuralları rahat yönetiyor, "keşke buna özel bir skill olsa" diyorsan; agent/skill mantığını merak ediyorsan Senior'sın.

Hedef: Proje-özel skill / agent / hook yazmak. Domain bilgisini kalıcı yapıya gömmek.

Kazanımlar

- ✓ Skill / agent / hook **oluşturur** (frontmatter, tetik, kısıt)
- ✓ Model katmanı seçer: haiku (mekanik) / sonnet (analiz) / opus (yüksek-risk)
- ✓ Salt-okuma vs yazma ayrımı uygular (denetçi agent yazamaz)
- ✓ Tier 3 işte plan yazar, agent'ları çağırır (code-review, security)
- ✓ Proje-özel kural dosyaları yazar

Skill oluşturma — anatomi

```
# .claude/skills/<ad>/SKILL.md
---
name: migration-writer
description: Idempotent SQL migration yazar. "migration yaz",
  "şema değiştir" denildiğinde tetiklenir. Yedek almadan silme YASAK.
allowed-tools: Read, Write, Edit, Grep # yetkiyi KISITLA
user-invocable: true
model: inherit
---
# Gövde: amaç, adımlar, kısıtlar, çıktı formatı
```

description = router. Tetik ifadelerini buraya gerçek cümlelerle göm — Claude ne zaman çağıracağını buradan anlar.

Agent oluşturma — model + salt-okuma

```
---
name: sql-sp-reviewer
description: SQL'in İŞ DOĞRULUĞUNU denetler (transaction, ledger).
  SP yazıldıktan sonra proaktif çağır. security-reviewer ayrı bakar.
tools: Read, Grep, Glob, Bash # Edit/Write YOK = salt-okuma
model: opus # sessiz-bozulma riski → en güçlü
---
```

Hook oluşturma — otomatik koruma

```
# .claude/hooks/delete-guard.sh - PreToolUse
cmd=$(echo "$input" | jq -r '.tool_input.command // ""')
if echo "$cmd" | grep -qE 'DELETE FROM (ledger|transactions)'; then
  echo "BLOKLANDI: finansal kayıt silme yasak." >&2
  exit 2 # exit 2 = tool'u durdur
fi
exit 0
```

Kayıt: `.claude/settings.json` → `PreToolUse` bloğuna ekle (matcher: Bash).

Pratik: Çoğu zaman kendin yazmazsın — Claude'a "şu skill'i oluştur, şunları bilsin, şu ifadelerle tetiklensin" dersin, o dosyayı yazar, sen onaylarsın.

Geçiş kapısı → L3: Kendi projende rahat araç üretiyor, başka devler "senin kuralların" diye soruyorsa; ekip standardını düşünmeye başladıysan Leader'sın.

Leader — "Ekip standardı kuruyorum"

Hedef: Template'i ekibe yaymak, yönetmek; karar ve onboarding'i sistematik kılmak.

Kazanımlar

- ✓ Yeni projeye template kurar: `bootstrap.ps1/.sh`
- ✓ Template iyileşince projelere yayar: `--update` (proje-özel'e dokunmaz)
- ✓ Evrensel vs stack vs proje kural ayrımını yönetir
- ✓ Mimari karar → ADR yazar; kararsızlıkta `council this`
- ✓ Agent fleet + hook ekosistemi stratejisi (faz-kapanış kapısı vb.)
- ✓ Yeni dev onboarding: hangi doc kime (acemi rehberi → bu rehber)

Template kurulum / güncelleme

```
# Yeni proje
pwsh bootstrap.ps1 -ProjectPath "D:\Dev\proje" -ProjectName "Proje" -Stack dotnet-mvc

# Template iyileşti → tüm projelere yay (proje-özel korunur)
for p in api web cli; do bash bootstrap.sh --path /d/Dev/$p --update; done
```

Stack seçimi

Stack	Ne zaman
<code>dotnet-mvc</code>	ASP.NET Core MVC + EF + SQL Server, Razor
<code>nodejs-typescript</code>	Node + TS (Express/Fastify/Next)
<code>python-generic</code>	Python (Django/FastAPI/Flask)
<code>none</code>	Stack-bağımsız, sadece universal + proje

Skill / agent / hook oluşturma — bu seviyede

Tekil değil **ekosistem**: birbirine atıf yapan skill çiftleri, faz-kapanış kapısı (build→review→security zinciri), takım çapında kural seti. Template'e generic katkı (PR).

Governance: Stale skill/kural = aktif zarar. Periyodik gözden geçir: "son 2 ayda tetiklendi mi, hâlâ doğru mu?" Hayır → emekli et/sil.

Geçiş kapısı → L4: Çok-ajanlı orkestrasyon, MCP entegrasyonu, ölçüm/otomasyon ihtiyacı doğduysa Plus'tasın.

Plus — "Sistemi otomatikleştiriyorum"

Hedef: Tek bağlama sığmayan işleri orkestre etmek; kendi araçlarını sisteme bağlamak.

Kazanımlar

- ✓ Workflow orkestrasyon: onlarca agent fan-out + adversarial verify
- ✓ MCP server yazar (kendi API/DB'ni Claude'a tool olarak bağlar)
- ✓ Çok-hook'lu pipeline (format + lint + test + bildirim)
- ✓ Zamanlanmış/loop görevler (cron routine, `/loop`)
- ✓ Meta: sistemin kendini iyileştirmesi, telemetri, ölçüm

Workflow orkestrasyon (kavram)

```
# Çok dosyalı review: her boyut bulur → her bulgu doğrulanır
pipeline(DIMENSIONS,
  d => agent(d.prompt, {schema, phase:'Review'}),
  review => parallel(review.findings.map(f =>
    agent('Adversarially verify: ${f}`, {phase:'Verify'}))))
```

Workflow = deterministik kontrol akışı (loop/fan-out/verify). Sadece kullanıcı açıkça isterse veya ultracode açıkken.

MCP — kendi tool'unu bağla

SQL Server, dosya sistemi, takvim, e-posta... Claude'a yeni yetenek = MCP server. Python (FastMCP) veya TS (MCP SDK). Bu repodaki `sqlserver-mcp-server` örnek.

Hook ekosistemi (ileri)

Olay	İleri kullanım
<code>PostToolUse</code> (Edit)	Otomatik format (prettier/dotnet format)
<code>Stop</code>	Yanıt bitince test/lint + bildirim
<code>UserPromptSubmit</code>	Bağlam enjeksiyonu, mesaj zenginleştirme

Plus zihniyeti: Artık tek tek işleri değil, *iş üreten sistemleri* tasarlıyorsun. Token bütçesi ve doğrulama (verify) birinci sınıf tasarım girdisi.

Aşırı-mühendislik tuzağı: Güçlü ≠ gerekli. Workflow/MCP pahalıdır. İhtiyaç kanıtlanmadan kurma — `coding-discipline` ilkesi burada da geçerli.

Hepsi tek sayfada

Seviye	Cümle	Oluşturma	Anahtar araç
L0 Acemi	Konuşabiliyorum	—	günaydın / iyi geceler
L1 Junior	Araçları kullanıyorum	"kurala yaz"	/code-check, /verify, compact/clear
L2 Senior	Araç üretiyorum	skill+agent+hook	frontmatter, model tier, plan-first
L3 Leader	Ekip standardı kuruyorum	ekosistem + template	bootstrap, ADR, council, governance
L4 Plus	Sistemi otomatikleştiriyorum	workflow + MCP	orquestrasyon, MCP, çok-hook

Üç değişmez ilke (her seviyede)

1 · Yazılı olan yaşar

Claude hatırlamaz. Kural/journal/dosya = kalıcı hafıza.

2 · Karar insanda

Claude uygular, sen onaylarsın. Gösterileni oku.

3 · Acıyı kalıcı yapıya çevir — Aynı şeyi 2. kez yaptıysan/anlattıysan: kural, skill, agent veya hook doğmalı. İlk kez yapma, ikinci kez doğur, üçüncü olmasın.

Daha derine

acemi_rehberi.pdf · ilk adımlar · GELISTIRICI_REHBERI.md · araç kataloğu · PROJE_OZEL_KAVRAM.md
neden/niçin · PROJE_OZEL_ORNEK_SENARYO.md · kopyala-yapıştır prompt · PROJE_OZEL_OLUSTURMA.md
teknik yazım · USAGE.md · bootstrap

Tek cümlelik yol: Kullan (L0-L1) → Üret (L2) → Standartlaştır (L3) → Otomatikleştir (L4).

claude-context-template · Ustalık Yolu · Sürüm 1.0